

**UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO**

Relatório do CRUD NoSQL – MongoDB

**ANTHONY LEVI TAVARES DOS SANTOS
ISAAC EVANGELISTA REIS**

São Cristóvão

12/07/26

1 INTRODUÇÃO

Este relatório apresenta a implementação das operações básicas de CRUD (Create, Read, Update e Delete) utilizando o banco de dados não relacional MongoDB. O objetivo do trabalho foi desenvolver métodos capazes de realizar inserção, consulta, atualização e remoção de documentos em coleções do banco de dados, permitindo a manipulação persistente das informações armazenadas.

A aplicação foi desenvolvida utilizando a linguagem Python, realizando a comunicação com o MongoDB por meio de operações específicas sobre os documentos armazenados. Para cada método implementado, são apresentados exemplos de execução e seus respectivos efeitos no banco de dados, evidenciando as alterações realizadas nas coleções.

2 MÉTODO DE INSERÇÃO

create(self, entity)

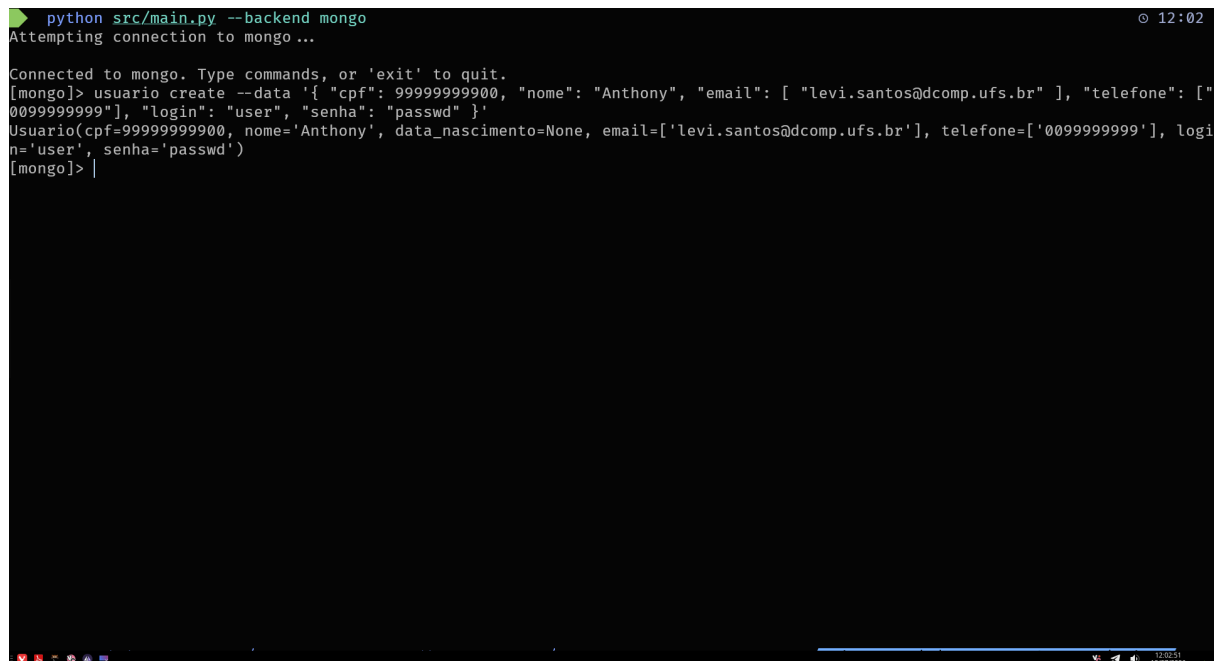
```
def create(self, entity: T) -> T:
    data = {k: v for k, v in asdict(entity).items() if v is not None}

    # Do not accept duplicates
    try:
        self.collection.insert_one(data)
    except DuplicateKeyError:
        raise DuplicateKeyException(self.entity_cls.__name__)

    return self._row_to_entity(data)
```

Responsável por inserir um novo documento na coleção do MongoDB a partir da entidade recebida como parâmetro. O método também verifica a existência de chaves duplicadas, impedindo a inserção de registros já existentes.

Efeito no banco:



```
python src/main.py --backend mongo
Attempting connection to mongo ...
Connected to mongo. Type commands, or 'exit' to quit.
[mongo]> usuario create --data '{ "cpf": 99999999900, "nome": "Anthony", "email": [ "levi.santos@dcomp.ufs.br" ], "telefone": [ "0099999999" ], "login": "user", "senha": "passwd" }'
Usuario(cpf=99999999900, nome='Anthony', data_nascimento=None, email=['levi.santos@dcomp.ufs.br'], telefone=['0099999999'], login='user', senha='passwd')
[mongo]> |
```

3 MÉTODO DE CONSULTA

get(self, key: dict)

```
def get(self, key: dict) -> T | None:
    doc = self.collection.find_one(key)
    return self._row_to_entity(doc) if doc else None
```

Realiza a busca de um documento na coleção utilizando a chave informada. Caso o documento seja encontrado, ele é convertido para a entidade correspondente e retornado ao usuário.

Efeito no banco:

```
v: 2,
key: { login: 1 },
name: 'login_1',
unique: true,
sparse: true
}
]
universidade> db.usuario.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { cpf: 1 }, name: 'cpf_1', unique: true },
  {
    v: 2,
    key: { login: 1 },
    name: 'login_1',
    unique: true,
    sparse: true
  }
]
universidade> db.usuario.getDB()
universidade
universidade> db.usuario.find()
[
  {
    _id: ObjectId('6a54fe13d1e40db34359885b'),
    cpf: Long('99999999900'),
    nome: 'Anthony',
    email: [ 'levi.santos@dcomp.ufs.br' ],
    telefone: [ '0099999999' ],
    login: 'user',
    senha: 'passwd'
  }
]
universidade>
```

Kitty - ~/P/EDPRJ: nvim mongosh mongodb://<credentials>@18... admin@ip-172-31-38-240: ~ / Kitty - ~/P/EDPRJ: python sr...

4 MÉTODO DE ATUALIZAÇÃO

update(self, key: dict, fields: dict)

```
def update(self, key: dict, fields: dict) -> T | None:
    try:
        result = self.collection.update_one(key, {"$set": fields})
    except DuplicateKeyError:
        raise DuplicateKeyException(self.entity_cls.__name__)

    if result.matched_count == 0:
        raise KeyError(f"{self.entity_cls.__name__} with key {key} not found")

    doc = self.collection.find_one(key)
    return self._row_to_entity(doc) if doc else None
```

Atualiza os campos especificados de um documento existente, identificado pela chave fornecida. Caso a chave não seja encontrada ou a atualização gere conflito de unicidade, uma exceção é lançada.

Efeito no banco:

```
python src/main.py --backend mongo
Attempting connection to mongo ...
Connected to mongo. Type commands, or 'exit' to quit.
[mongo]> usuario update --key '{ "cpf": 99999999900 }' --fields '{ "email": [ "levi.santos@dcomp.ufs.br", "acube_one@disroot.org" ] }'
Usuario(cpf=99999999900, nome='Anthony', data_nascimento=None, email=['levi.santos@dcomp.ufs.br', 'acube_one@disroot.org'], telefone=['0099999999'], login='user', senha='passwd')
[mongo]>

Kitty - ~/P/EDPRJ: nvim / mongosh mongodb://<credentia... / admin@ip-172-31-38-240: ~ Kitty - ~/P/EDPRJ: python src/main...
```

```

v: 2,
key: { login: 1 },
name: 'login_1',
unique: true,
sparse: true
}
]
universidade> db.usuario.getDB()
universidade
universidade> db.usuario.find()
[
  {
    _id: ObjectId('6a54fe13d1e40db34359885b'),
    cpf: Long('99999999900'),
    nome: 'Anthony',
    email: [ 'levi.santos@comp.ufs.br' ],
    telefone: [ '0099999999' ],
    login: 'user',
    senha: 'passwd'
  }
]
universidade> db.usuario.find()
[
  {
    _id: ObjectId('6a54fe13d1e40db34359885b'),
    cpf: Long('99999999900'),
    nome: 'Anthony',
    email: [ 'levi.santos@comp.ufs.br', 'acube_one@disroot.org' ],
    telefone: [ '0099999999' ],
    login: 'user',
    senha: 'passwd'
  }
]
universidade> |
Kitty - ~/EDPRJ: nvim mongosh mongodb://<credentials>@18... admin@ip-172-31-38-240: ~ / Kitty - ~/EDPRJ: python sr...

```

5 MÉTODO DE REMOÇÃO

delete(key)

```
def delete(self, key: dict) -> bool:
    self._check_no_references(key)

    result = self.collection.delete_one(key)
    return result.deleted_count > 0
```

Remove um documento da coleção com base na chave informada. Antes da exclusão, o método verifica se existem outros documentos que dependem daquele registro, evitando a remoção de dados ainda referenciados.

Efeito no banco:

```
python src/main.py --backend mongo
Attempting connection to mongo ...
Connected to mongo. Type commands, or 'exit' to quit.
[mongo]> usuario delete --key '{"cpf": 99999999900}'
True
[mongo]> |
```

Kitty - ~/P/EDPRJ: nvim / mongosh mongodb://<credentia... / admin@ip-172-31-38-240: ~ Kitty - ~/P/EDPRJ: python src/main...

```
name: 'login_1',
unique: true,
sparse: true
}
]
universidade> db.usuario.getDB()
universidade
universidade> db.usuario.find()
[
  {
    _id: ObjectId('6a54fe13d1e40db34359885b'),
    cpf: Long('99999999900'),
    nome: 'Anthony',
    email: [ 'levi.santos@dcomp.ufs.br' ],
    telefone: [ '0099999999' ],
    login: 'user',
    senha: 'passwd'
  }
]
universidade> db.usuario.find()
[
  {
    _id: ObjectId('6a54fe13d1e40db34359885b'),
    cpf: Long('99999999900'),
    nome: 'Anthony',
    email: [ 'levi.santos@dcomp.ufs.br', 'acube_one@disroot.org' ],
    telefone: [ '0099999999' ],
    login: 'user',
    senha: 'passwd'
  }
]
universidade> db.usuario.find()
universidade>
Kitty - ~/P/EDPRJ: nvim /mongosh mongodb://<credentials>@18... admin@ip-172-31-38-240: ~ / Kitty - ~/P/EDPRJ: python sr...
```

6 MÉTODO DE LISTAGEM FILTRADA

`list(self, filters: dict = {})`

```
def list(self, filters: dict = {}) -> list[T]:
    docs = self.collection.find(filters)
    return [self._row_to_entity(d) for d in docs]
```

Retorna uma lista de documentos da coleção que satisfazem os filtros especificados. Caso nenhum filtro seja informado, todos os documentos da coleção são retornados.

Efeito no banco:

```
python src/main.py --backend mongo
Attempting connection to mongo ...
Connected to mongo. Type commands, or 'exit' to quit.
[mongo]> usuario list
[Usuario(cpf=99999999900, nome='Anthony', data_nascimento=None, email=['levi.santos@dcomp.ufs.br', 'acube_one@disroot.org'], telefone=['0099999999'], login='user', senha='passwd')]
[mongo]>
```

Kitty - ~/P/EDPRJ: nvim / mongosh mongodb://<credentia... / admin@ip-172-31-38-240: ~ Kitty - ~/P/EDPRJ: python src/main...